

Python DA Basics Week 7 Quiz

1. What will be the output of the following code?

1점

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Name": ["Amy", None, "Chris", "Dana"],
    "Math": [90, 80, np.nan, 70],
    "City": ["Seoul", "Busan", "Daegu", None]
})

print(df.isna().sum().tolist())
```

한 개의 타원형만 표시합니다.

- ☐ [0, 1, 1]
- ☐ [1, 1, 1]
- ☐ [1, 2, 1]
- ☐ 옵션 4
- ☐ [3, 1, 0]

2. Which statement is CORRECT?

1점

```
df = pd.DataFrame({
    "student_id": [101, 102, 102, 103, 103],
    "name": ["Amy", "Ben", "Ben", "Chris", "Chris"],
    "score": [90, 80, 80, 70, 75]
})
```

한 개의 타원형만 표시합니다.

- ☐ df.duplicated().sum() returns 2, because both student IDs 102 and 103 occur more than once.
- ☐ df.drop_duplicates() removes both second appearances of student IDs 102 and 103 because duplicated student IDs are sufficient for complete-row duplication.
- ☐ df.drop_duplicates(subset=["student_id"]) keeps one row for each student ID, so only three rows remain.
- ☐ df.drop_duplicates(subset=["student_id"]) always keeps the row with the highest score for each student automatically.

3. What will be the output?

1점

```
import pandas as pd
import numpy as np

s = pd.Series([
    "90",
    "75",
    "not available",
    "88"
])

result = pd.to_numeric(
    s,
    errors="coerce"
)

print(result.isna().sum())
```

한 개의 타원형만 표시합니다.

- ☐ 0
- ☐ 1
- ☐ 2
- ☐ Runtime Error

4. Which statement is INCORRECT?

1점

한 개의 타원형만 표시합니다.

- ☐ str.strip() removes leading and trailing spaces from string values but does not normally remove spaces inside the string.
- ☐ str.upper() and str.title() can help standardize inconsistent capitalization before later analysis.
- ☐ replace("not available", np.nan) can be used to convert a special text value into a missing value before numeric conversion.
- ☐ astype(float) is generally safer than pd.to_numeric(errors="coerce") when a column contains both numeric strings and unknown nonnumeric text, because astype(float) automatically converts invalid values into NaN.

5. A data analyst is cleaning and analyzing the following student dataset.

3점

Which explanation MOST ACCURATELY describes the final output and the reasoning behind it?

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name": [" Amy ", "Ben", "Ben", "Chris", "Dana"],
    "class": ["a", "B", "B", "A", "C"],
    "math": ["90", "80", "80", "unknown", "70"],
    "english": [85, 90, 90, 95, np.nan]
})

cleaned = df.copy()

cleaned["name"] = cleaned["name"].str.strip()
cleaned["class"] = cleaned["class"].str.upper()

cleaned["math"] = pd.to_numeric(
    cleaned["math"],
    errors="coerce"
)

cleaned = cleaned.drop_duplicates()

cleaned["english"] = cleaned["english"].fillna(
    cleaned["english"].mean()
)

cleaned["average"] = (
    cleaned["math"] + cleaned["english"]
) / 2

cleaned["result"] = np.where(
    cleaned["average"] >= 85,
    "PASS",
    "FAIL"
)

selected = cleaned.loc[
    cleaned["class"].isin(["A", "B"])
    & cleaned["average"].between(80, 95)
]

print(selected["name"].tolist())
```

한 개의 타원형만 표시합니다.

- ☐ The final list is ['Amy', 'Ben', 'Chris']. Amy remains because trimming spaces changes only the text representation, Ben appears once because the two Ben rows become exact duplicates after the class values are standardized and are therefore reduced by drop_duplicates(), and Chris is included because converting "unknown" with errors="coerce" produces NaN, which is automatically ignored when calculating the row average. Dana is excluded because class C is not included in the final isin() condition.

- ☐ The final list is ['Amy', 'Ben']. Amy has an average of 87.5 and satisfies both the class and average conditions. The two Ben records become identical after cleaning and only one remains, with an average of 85. Chris has a missing Math value after numeric conversion, so arithmetic with that NaN produces a missing average and the `between()` condition does not select the row. Dana's missing English value is filled with the column mean, but class C prevents Dana from entering the final result.
- ☐ The final list is ['Amy', 'Ben', 'Chris', 'Dana']. `drop_duplicates()` removes only duplicated index values rather than duplicated row contents, so both Ben rows are preserved until the final filter. Chris qualifies because `pd.to_numeric(errors="coerce")` replaces invalid numeric strings with zero, and Dana qualifies because `fillna(mean)` makes every originally incomplete row automatically satisfy `between(80, 95)`.
- ☐ The final list is ['Ben']. Amy is removed because `str.strip()` creates a modified Series without changing the DataFrame unless `inplace=True` is specified. Ben survives because the duplicated records are reduced to one row. Chris is removed after "unknown" becomes NaN, and Dana is removed because the class condition excludes C.

6. Consider the following code.
What will be printed?

1점

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "id": [1, 2, 2, 3, 4, 5],
    "name": [" Amy ", "Ben", "Ben", "Chris", None, " Dana "],
    "city": ["seoul", "BUSAN", "BUSAN", " daegu ", "Seoul", None],
    "math": [90, 80, 80, 999, 70, np.nan],
    "science": ["85", "90", "90", "75", "not available", "95"]
})

cleaned = df.copy()

cleaned = cleaned.drop_duplicates(subset=["id"])

cleaned["name"] = cleaned["name"].str.strip()
cleaned["city"] = cleaned["city"].str.strip().str.title()

cleaned["science"] = pd.to_numeric(
    cleaned["science"],
    errors="coerce"
)

cleaned.loc[
    ~cleaned["math"].between(0, 100),
    "math"
] = np.nan

cleaned = cleaned.dropna(subset=["name"])

print(cleaned["id"].tolist())
print(cleaned["science"].isna().sum())
print(cleaned["math"].isna().sum())
```

한 개의 타원형만 표시합니다.

- ☐ [1,2,3,5] 0 2
- ☐ [1,2,3,5] 1 2
- ☐ [1,2,3,4,5] 1 2
- ☐ [1,2,3,5] 0 1

7. A university receives a student dataset. Before cleaning, the analyst observes:

2점

Shape: (9, 7)

Missing values:

student_id 0

name 1

class 0

city 1

math 1

english 1

science 0

The analyst also discovers:

- Two rows share the same student_id and contain the same values.
- One Math value is 999.
- One Science value is the string "not available".
- Several name and city values contain leading or trailing spaces.
- One class value is "c" while the intended standardized form is "C".

```

cleaned = dirty_students.copy()

cleaned = cleaned.drop_duplicates(
    subset=["student_id"]
)

cleaned["name"] = cleaned["name"].str.strip()
cleaned["city"] = cleaned["city"].str.strip()

cleaned["class"] = cleaned["class"].str.upper()
cleaned["city"] = cleaned["city"].str.title()

cleaned["science"] = pd.to_numeric(
    cleaned["science"],
    errors="coerce"
)

cleaned.loc[
    ~cleaned["math"].between(0, 100),
    "math"
] = np.nan

cleaned = cleaned.dropna(
    subset=["name"]
)

cleaned["city"] = cleaned["city"].fillna(
    "Unknown"
)

for col in ["math", "english", "science"]:
    cleaned[col] = cleaned[col].fillna(
        cleaned[col].mean()
    )

```

한 개의 타원형만 표시합니다.

- ☐ The pipeline is fundamentally incorrect because `drop_duplicates()` must always be the final step; removing duplicates before filling missing values causes Pandas to lose track of the original row indices and prevents subsequent string or numeric operations.
- ☐ The pipeline is appropriate for the stated cleaning policy: it preserves the original DataFrame, removes duplicate students by ID, standardizes text, converts invalid Science text to missing values, converts invalid Math values to missing values, removes rows without names, fills missing cities separately, and only then calculates means for the remaining missing numeric scores.

- ☐ The pipeline is incorrect primarily because `between(0, 100)` identifies invalid Math scores. Therefore the Boolean condition should be used directly without `~`, and valid scores should be replaced by NaN so that only abnormal values remain for analysis.
- ☐ The pipeline is inappropriate because `pd.to_numeric(errors="coerce")` should be applied only after all missing numeric values have already been filled; otherwise "not available" remains a string and cannot participate in the mean calculation.

이 콘텐츠는 Google이 만들거나 승인하지 않았습니다.

Google 설문지